COURSE DESCRIPTION:

HTML (Hypertext Markup Language)

Course Description: This course introduces students to HTML(Hypertext Markup Language),the standard language for creating web pages. The course covers fundamental topics such as structure and syntax of HTML, elements, attributes, and tags. Students will learn to create static web pages using HTML and understand the basics of web design principles. Additionally, the course explores advanced concepts like HTML5 semantic elements, multimedia embedding, and responsive design techniques. By the end of the course, students will be able to build, format, and structure web pages that can be viewed across different browsers and devices.

Office Automation and Information Technology

Course Description: This course provides an introduction to office automation tools and information technology fundamentals. Students will learn to use essential office productivity software such as word processors, spreadsheets, presentation software, and email management tools. The course also covers topics in information technology (IT) including hardware, software, operating systems, and networking concepts. Key modules include document formatting, data analysis using spreadsheets, preparing presentations, and collaboration tools. Students will develop the technical skills necessary for efficient office operations and gain a solid foundation in IT for professional and personal productivity.

Data Warehousing

Course Description: The Data Ware housing course explores the principles and practices involved in managing large-scale data repositories, designed for analytical processing. Students will be introduced to the architecture, design, and implementation of data warehouses. Topics include data modeling, ETL (Extract, Transform, Load) processes, OLAP (Online Analytical Processing), data mining, and performance tuning. The course emphasizes real-world applications of data warehousing techniques to store and analyze large amounts of business data, facilitating decision-making processes. Students will also gain an understanding of the integration of data from various sources into a central repository, and the role of data warehousing in business intelligence systems.

Database Management Systems (DBMS)

Course Description: This course provides an in-depth understanding of database management systems (DBMS), focusing on the design, development, and management of databases. Topics include relational databases, data models, normalization, SQL (Structured Query Language), transaction management, indexing, and database security. Students will learn how to design efficient database structures, perform complex queries, and ensure the integrity and consistency of data in a multi-user environment. The course also covers advanced DBMS concepts like distributed databases, cloud databases, and NoSQL technologies. Practical exercises involve working with popular DBMS platforms like MySQL, PostgreSQL, and Oracle

OBJECTIVES OF THE INTERNSHIP IN COMPUTER SCIENCE

An internship in computer science provides students with hands-on experience and practical knowledge in the field of technology. The primary objectives of such an internship include:

1. **Application of Theoretical Knowledge:**
    - Enable students to apply the concepts, theories, and techniques learned in their academic courses to real-world projects and problems.
    - Develop a deeper understanding of various computer science topics such as programming, software development, algorithms, and systems analysis.
2. **Skill Development:**
    - Enhance technical skills in programming languages (e.g, Python, Java, C++) and tools used in the industry.
    - Gain proficiency in software development practices, debugging, and problem-solving.
    - Learn about industry-standard software, frame works, and tools, including version control (e.g., Git), databases, and integrated development environments (IDEs).
3. **Exposure to Industry Practices:**
    - Provide insights in to the work flow, practices, and methodologies used in the tech industry (e.g., Agile, Scrum, DevOps).
    - Understand the software development lifecycle (SDLC) and work collaboratively within a development team.
4. **Communication and Collaboration:**
    - Improve interpersonal and professional communication skills, particularly in presenting ideas and collaborating with peers and mentors.
    - Work in a team environment to solve problems, contribute to project development, and engage in code reviews.
5. **Problem Solving and Critical Thinking:**
    - Encourage independent thinking by tackling real-world challenges and providing innovative solutions.
    - Develop strong analytical skills and enhance the ability to break down complex problems into manageable components.
6. **Problem Solving and Critical Thinking:**
    - Encourage independent thinking by tackling real-world challenges and providing innovative solutions.

- Develop strong analytical skills and enhance the ability to break down complex problems into manageable components.

7. **Networking and Professional Growth:**
   - Foster professional relationships with industry mentors, colleagues, and other professionals in the field.
   - Gain exposure to potential employers and explore career opportunities in the computer science and technology sectors.
8. **Exposure to Emerging Technologies:**
   - Gain experience with cutting-edge technologies like artificial intelligence (AI), machine learning (ML), block chain, data science, and cloud computing.
   - Develop an understanding of current trends and innovations in the technology industry.
9. **Career Exploration and Focus:**
   - Help students explore various career paths in computer science, such as software development, systems administration, data analysis, and cyber security.
   - Assist studentsindeterminingtheirareaofinterestorspecializationwithincomputer science.
10. **Port folio Development:**
    - Allow students to work on projects that can be added to their personal port folios, show casing their technical skills and achievements to future employers.
    - Build a solid track record of real-world projects that demonstrate competence in computer science and technology.
11. **Feed back and Improvement:**
    - Receive constructive feedback from supervisors and mentors to improve technical abilities and professional conduct.

IDENTIFY STRENGTHS AND AREAS OF IMPROVEMENT, WHICH CAN HELP IN

MAKING INFORMED DECISIONS ABOUT FURTHER CAREER DEVELOPMENT OR ACADEMIC FOCUS. CHOICE OF WORKPLACE FOR INTERNSHIP IN COMPUTER SCIENCE

Choosing the right workplace for an internship in computer science is crucial for gaining valuable experience, skill development, and career progression. The ideal workplace depends on various factors such as the type of experience you're looking for, the skills you want to develop, and the overall career path you intend to follow. Below are some key considerations to guide you in selecting the right workplace for your computer science internship:

1. **Type of Organization:**

- **Tech Companies (Startups, Mid-size, or Large Enterprises):**
  - **Why choose:** Tech companies offer exposure to cutting-edge technologies, software development practices, and large-scale projects. Interns often work on real-world problems and collaborate with professionals in areas such as AI, machine learning, cloud computing, and cyber security.
  - **Example:** Google, Microsoft, Amazon, small local tech startups, or established software firms.
- **Software Development Agencies:**
  - **Why choose:** These agencies often work on a variety of projects for clients in different industries. It's a great place to get hands-on experience with client-oriented solutions, agile methodologies, and multi-faceted software development.
  - **Example:** Agencies that specialize in web development, app development, or custom software solutions.
- **Research Labs/Institutes:**
  - **Why choose:** If you're interested in the academic or research side of computer science, internships at research labs offer opportunities to engage in cutting-edge research in areas like artificial intelligence, data science, robotics, and cyber security.
  - **Example:** University research labs or private tech research organizations.
- **Financial or Health care Organizations:**
  - **Why choose:** Many large financial and healthcare institutions require computer science professionals for data analysis, software systems development, and security
  - **solutions. Interning in these sectors can provide exposure to highly specialized software applications and data-driven decision-making.**

Example: Banks (e.g., JPMorgan), health care institutions (e.g., Mayo Clinic), or insurance firms (e.g., Allianz).

PROFILE OF THE WORK PLACE: ST.PHILOMENA'S (AUTONOMOUS) COLLEGE, MYSORE

Institution Overview: St. Philomena's Autonomous College, Mysore, is a renowned educational institution that offers a diverse range of undergraduate and postgraduate programs in arts, sciences, commerce, and computer science. It is an autonomous college, which gives it the flexibility to design its curriculum and academic structure while adhering to quality education standards set by the University Grants Commission (UGC) and affiliating universities.

The college is located in the heart of Mysore and is known for its academic excellence, infrastructural facilities, and vibrant campus life. The Computer Science department at St. Philomena's (Autonomous) College is well-regarded for providing both theoretical and practical knowledge in the rapidly evolving field of technology.

ORGANIZATIONAL STRUCTURE FOR AN INTERNSHIP PROGRAM.

1. **Intern:**
    - Responsible for completing tasks, learning, and contributing to projects. Works closely with supervisors and mentors for guidance and feedback.
2. **Supervisor (Internship Manager/ TeamLead):**
    - Oversees the intern's day-to-day work, assigns tasks, and provides performance feedback. Ensures learning objectives are met.
3. **Mentor:**
    - Offers guidance on career development, technical skills, and work place integration. Provides informal support and advice.
4. **Internship Program Coordinator/ HR Team:**
    - Manages the internship process, including recruitment, on boarding, and administrative tasks. Ensures smooth integration of the intern into the company.
5. **Department/Team:**
    - The specific team or department (e.g., software development or data science) the intern works in. Provides the intern with tasks and projects related to their field.
6. **Senior Management (Optional):**
    - Occasionally interacts with interns for presentations or evaluations. Ensures the internship aligns with organizational goals.
7. **Peer Interns:**
    - Other interns who work alongside and collaborate with the intern on related projects.

NATURE AND MODE OF INTERNSHIP WORK

The nature and mode of internship work depend on the specific goals of the internship, the organization, and the field of study. In the context of a Computer

Science Internship, the work generally follows one of the following formats:

1. Project-Based Work:

- **Nature:** Interns are assigned to work on specific projects, which could include tasks like developing a software application, testing software, or creating data models. These projects are usually aligned with the company's ongoing work and often have clear deliverables.
- **Mode:** Interns typically work independently or in small teams, with regular check-ins to discuss progress and receive feedback.
- **Example:** Developing a mobile app, designing a website, or creating an algorithm.

2. Research-Oriented Work:

- **Nature:** Interns might be involved in research tasks that help the company or institution solve specific problems or explore new technologies. This type of work is common in academic or research-based organizations.
- **Mode:** The intern conducts literature reviews, gathers data, analyzes findings, and helps draft research papers or reports.
- **Example:** Research in areas like artificial intelligence, machine learning, or block chain technology.

3. Support and Maintenance:

- **Nature:** Interns assist with the maintenance and support of existing systems, software, or IT infrastructure. This includes bug fixing, system upgrades, troubleshooting, or performing routine tasks to ensure systems run smoothly.
- **Mode:** Interns often work under the supervision of senior developers or IT administrators, gaining hands-on experience with operational systems.
- **Example:** Debugging code, monitoring server performance, or updating databases.

4. Software Development:

- **Nature:** Interns engage in hands-on software development, which may involve writing code, developing features, debugging, and testing. Interns contribute to ongoing development projects and often collaborate with other team members.
- **Mode:** The work is typically done in collaboration with a development team, using version

control tools like Git for project management and communication.

- **Example:** Coding in languages like Java, Python, or C++ for software or applications.

5. Data Analytics and Data Science:

- **Nature:** Interns may be tasked with analyzing large data sets, extracting meaningful insights, or building predictive models. This involves working with data analytics tools and programming languages like Python, R, or SQL.
- **Mode:** Interns typically use data sets provided by the company, applying data analysis techniques to derive actionable insights and reports.
- **Example:** Analyzing customer data, creating data visualizations, or building machine learning models.

CHALLENGESFACEDBYSTUDENTSIN INTERNSHIPS

Internships offer valuable learning experiences but can also present challenges that students need to navigate. These challenges vary depending on the type of internship, the industry, and the specific company, but common issues faced by students during internships in fields like computer science include:

1. **Lack of Practical Experience:**

- **Challenge:** Many students face difficulties when transitioning from theoretical learning in the class room to real-world work environments. They may lack experience with practical tools, software, and technologies used in the industry.
- **Impact:** It can lead to feelings of in adequacy or frustration, especially when they are unable to immediately contribute to complex tasks.
- **Solution:** Interns can overcome this by being proactive in seeking help, using online resources for self-learning, and asking for more manageable tasks initially to build their confidence.

2. **Communication Issues:**

- **Challenge:** Interns, especially remote ones, may struggle with communication barriers, whether it's with team members, supervisors, or other departments. They may feel disconnected or unsure about how to communicate effectively in a professional setting.
- **Impact:** Poor communication can lead to misunderstandings, delays in completing tasks, and missed learning opportunities.
- **Solution:** Interns should actively seek clarification on assignments, schedule regular check-ins with their supervisors, and make an effort to participate in team meetings. Clear and open communication is key.

3. **Balancing Internship with Studies:**

- **Challenge:** Students often juggle internships with ongoing academic commitments. Managing time effectively between work responsibilities and course work can become overwhelming.
- **Impact:** Stress, burn out, and lower academic performance are potential consequences.
- **Solution:** Effective time management is essential. Creating a schedulethatallocatestimefor work, study, and rest, along with setting clear priorities, helps in maintaining a balance.

4. **Limited Guidance or Mentorship:**

- **Challenge:** Some interns experience a lack of mentorship or guidance from their supervisors. Without proper direction, they may feel uncertain about their tasks or struggle to develop their skills.
- **Impact:** This lack of support can hinder their learning and reduce the quality of work they produce.
- **Solution:** Interns should take the initiative to seek feedback regularly, ask questions, and request guidance when needed. Establishing a good relationship with a mentor can greatly

enhance the internship experience.

5. **Difficulty Adapting to Workplace Culture:**
- **Challenge:** Interns may find it difficult to adapt to the workplace culture, especially if they are working in a large organization with a formal or fast-paced environment. This includes understanding office etiquette, communication styles, and the organizational structure.
- **Impact:** Feeling out of place or uncomfortable can affect an intern's confidence and performance.
- **Solution:** Observing the workplace dynamics, engaging with colleagues, and asking questions about the culture can help interns integrate more smoothly. Being open to learning and adapting is crucial.

6. **Technical Skills Gap:**
- **Challenge:** In fields like computer science, there may be a gap between what students have learned in school and the tools, programming languages, or frameworks required in their internships. For example, knowledge of specific programming languages or technologies may not always align with the internship requirements.
- **Impact:** Interns might struggle to keep up with technical tasks, resulting in slower progress and frustration.
- **Solution:** Before starting the internship, students can review the required tools and technologies. During the internship, they should seek out additional training or resources to quickly get up to speed.

7. **Limited Responsibility or Repetitive Tasks:**
- **Challenge:** Some interns find that the tasks they are assigned are either too menial(e.g., data entry, document formatting) or do not challenge them enough to develop new skills.
- **Impact:** This can result in dissatisfaction and a lack of motivation.
- **Solution:** Interns can request more challenging assignments or inquire about contributing to larger projects. It's also an opportunity to learn how even small tasks contribute to the broader goals of the company.

LEARNING OUTCOMESOFAN INTERNSHIP

1. **Practical Application of Knowledge:**
    - Interns will apply theoretical concepts learned in the classroom to real-world problems, gaining hands-on experience in their field of study, such as computer science, software development, or data analysis.
2. **Development of Technical Skills:**
    - Internswillenhancetheirtechnicalabilitiesbyworkingwithindustry-standardtools, programming languages, software, and technologies, bridging the gap between

academic knowledge and industry requirements.

3. **Problem-Solving and Critical Thinking:**
    - Interns will improve their problem-solving skills by tackling real challenges and finding innovative solutions. They will also develop critical thinking abilities while working on tasks that require analysis and decision-making.
4. **Communication and Collaboration:**
    - Interns will enhance their communication skills by collaborating with teams, presenting ideas, and engaging with supervisors and mentors. They will learn how to effectively communicate technical information to both technical and non-technical audiences.
5. **Professionalism and Workplace Etiquette:**
    - Interns will gain a deeper understanding of professional behavior, time management, workplace etiquette, and the importance of maintaining a positive attitude and work ethic in a professional setting.
6. **Networking and Career Development:**
    - Interns will have opportunities to build professional networks by connecting with colleagues, mentors, and industry professionals. This will help in career development and open doors for future job opportunities.
7. **Understanding of Industry Practices:**
    - Interns will gain insight into industry-specific workflows, tools, methodologies, and practices. They will develop a deeper understanding of how organizations operate and how their roles fit into larger projects and goals.
8. **Feedback and Self-Improvement:**
    - Interns will receive constructive feedback from supervisors and peers, which will help them identify areas for improvement. This feedback fosters personal and professional growth and prepares interns for future career success.
9. **Increased Employability:**
    - Through the internship experience, interns will enhance their resumes with real-world work experience, boosting their chances of securing future employment opportunities in their field.
10. **Exposure to Emerging Technologies:**
    - Interns will be exposed to the latest tools, technologies, and industry trends, expandingtheirknowledgeandpreparingthemforfuturedevelopmentsintheir profession.

CONCLUSION & RETROSPECTION

In conclusion, an internship is an essential experience that bridges the gap between theoretical knowledge and practical application. It provides students with a firsthand understanding of their field, be it computer science, engineering, business, or any other discipline, allowing them to apply what they have learned in real-world scenarios. Interns gain valuable hands-on experience, develop technical skills, and learn how to navigate workplace dynamics. They also grow personally, improving communication, time management, and problem-solving abilities.

Reflecting on the internship experience, students often recognize significant growth whether it's in technical proficiency, professional behavior, or confidence in their abilities. They also gain insight into the specific tools, technologies, and methodologies used in their industry, which not only enhances their skill set but also prepares them for future career opportunities. The challenges faced during the internship, such as adapting to a new environment, managing workloads, and overcoming technical gaps, contribute to the intern's overall development and resilience**.**

Ultimately, internships provide a comprehensive learning experience that helps shape a student's career trajectory. The exposure to industry professionals, networking opportunities, and real-time feedback contribute to both personal and professional development, making internships a crucial phase in preparing for a successful career. This experience not only equips interns with the skills and knowledge needed to excel but also boosts their confidence and employability, offering a strong foundation for future success in their chosen fields.

APPENDICES

Appendices in an internship report or document typically include supplementary materials that support the content of the main report. These may include additional data, charts, documents, or examples that provide further detail but are not essential to understanding them in text. Below are some examples of what could be included in the appendices:

Appendix A: Internship Tasks and Responsibilities

- **Detailed List of Tasks:**

- A list of the key tasks and projects undertaken during the internship.
- Descriptions of responsibilities, including any software or tools used.
- Any specific milestones or achievements related to the internship.

Appendix B: Weekly Progress Reports

- **Weekly Summary:**
  - A brief summary of what was accomplished each week during the internship.
  - Any challenges faced and how they were addressed.
  - Feedback or insights from the supervisor.

Appendix C: Feedback from Supervisor

- **Supervise or Evaluation:**
  - A copy of the formal or informal evaluation provided by the internship supervisor at the end of the internship.
  - Includes feedback on performance, skills developed, and areas for improvement.

Appendix D: Technical Skills and Tools

- **List of Tools and Technologies Used:**
  - A detailed list of the software, programming languages, or technologies used during the internship.
  - Includes any new tools learned or applied during the internship.
  - Examples of projects or tasks that involved these tools.

Appendix E: Learning Outcomes Assessment

- **Self-Assessment:**
  - A reflection or self-assessment of the learning outcomes achieved during the internship.
  - How the internship helped meet personal and professional goals.
  - Skills gained and how they relate to future career development.

Appendix F: Networking and Career Development Activities

- **Professional Networking:**

- Details of any networking events, meetings with industry professionals, or career development activities attended during the internship.
- Any professional connections made and their potential impact on career prospects.

Appendix G: Internship Certificate

- **Certificate of Completion:**
  - A copy of the official certificate or letter confirming the completion of the internship, if applicable.

Appendix H: Supporting Documents and References

- **Supporting Documents:**
  - Any additional documents such as reports, presentations, or technical documentation produced during the internship.
  - References to key research or articles used during the internship.

HISTORYOFCOMPUTER

a. ABACUS: The history of computer begins with the birth of abacus which is believed to be the first computer. It is said to beinventedaround4000yearsago.

b. NAPIER'SBONE:(1550-1617)

It is manually operated calculating device. It was invented by John Napier in the year(1550-1617). It is a device used to calculate using 9 different strips or bines marked with the numbers, so that the tools knows as Napier Bones**.**

c. PASCALINE:(1642-1644)

It is also known as arithmetic machine or adding machine. It was invented in the year(1642-1644) by the French mathematician and a philosopher named Pascal. It

is believed that it is the first mechanical and automatic calculator.

d. STEPPED-RECKONER:(1673)

A Stepped-Reckoner is a device that was developed by German mathematician and philosopher named William in the year 1673. It was also known as digital, mechanical calculator which was called Stepped-Reckoner.

e. DIFFERENCEENGINE:(1820)

It was invented by Charles Babbage; also know to be "The Father of Computer".

f. ANALYTICALENGINE:(1850)

It was invented by Charles Babbage, they were using punch cards.

g. TABULATING MACHINE: (1890) They were using punch cards.

GENERATION OF COMPUTER

- **FIRSTGENERATION (1946-1959)**

a) LargeinSize.

b) Lot ofHeatwas produced.

c) Not Portable.

d) TheI Generationwasintroducedintheyear(1946-1959)

e) Itwasveryslow,hugeand expensive.

f) Thesearethecomputersinwhichvacuumtubeswereasbasiccomponent of CPU and memory.

g) Thepunchcardswereusedasinputsand magnetictapesasoutputs.

Examples:

a. ENIAC: Electronic Discrete variable Automatic Computer

b. UNIVAC: Universal Automatic Computer

c. IBM-701

d. IBM-650

- **SECOND GENERATION (1959-1965)**

a) Transistors were used instead of vacuum tubes.

b) Over all it was better than first generation of computers.

Examples:

a. IBM -120

b. IBM –7094

c. UNIVAC1108

- **THIRD GENERATION (1965-1975)**

a) Instead of Transistor; IC's were used.

b) Compare to first and second generation; third generations were better.

Examples:

a. IBM-360

b. PDP-Personal Data Processor

c. IBM-570/168

d. IDC316

- **FOURTH GENERATION (1971-1980)**

a) Cheaper and Portable.

b) Programming Languages were introduced like C, C++.

Examples:

a. DEC10
b. STAR 100
c. PDP11
d. CRAY-1(Super Computer)
e. CRAY-X-MD (Super Computer)

- **FIFTH GENERATION: (1980- Present)**
  - Desktop, Laptops, Notebooks.
  - VLSI- Very Large Scale Integrated Circuit.
  - ULSI –Ultra Large Scale Integrated Circuit.

2. MICROSOFT-EXCEL

FORMULAE:

i. Total:=Sum(B2:H2)
ii. Minimum:=Min(B2:H2)
iii. Maximum:=Max(B2:H2)
iv. Average:=Average(B2:H2)
v. Count:=Count(B2:H2)
vi. Power: =Power(I2/100,2)
vii. Ceiling:=Ceiling(N2,1)
viii. Floor:=Floor(N2,1)
ix. Concatenate:=Concatenate(A2,F2)
x. Upper Case: =Upper(A2)
xi. Lower Case:=Lower(A2)
xii. Proper:=Proper(A2)
xiii. Date & Time: =Now()
xiv. Seconds:=Second(Now())

TABLE WORKSHEET

Name	M1	M2	M3	M4	M5	M6	M7	Total
Sabastine	100	95	98	95	99	96	98	681
Brunda	98	66	78	99	77	85	77	580
Celina	88	56	45	23	78	56	45	391
Bhoomika	99	89	79	98	99	68	78	610
Nafida	65	98	78	54	65	56	65	481
Netali	78	89	89	55	98	45	65	519
Jayshreee	89	69	78	45	69	88	78	516

Max	Min	Avg	Count	Power	Ceiling	Floor	CONCATENATE
100	95	97.285714	7	46.3761	47	46	Sabastine99
99	66	82.857143	7	33.64	34	33	Brunda77
88	23	55.857143	7	15.2881	16	15	Celina78
99	68	87.142857	7	37.21	38	37	Bhoomika99
98	54	68.714286	7	23.1361	24	23	Nafida65
98	45	74.142857	7	26.9361	27	26	Netali98
89	45	73.714286	7	26.6256	27	26	Jayshreee69

Uppercase	Lowercase	Proper	Date &Time	Seconds
SABASTINE	sabastine	Sabastine	4/2/20252:20	31
BRUNDA	brunda	Brunda	4/2/20252:20	31
CELINA	celina	Celina	4/2/20252:20	31
BHOOMIKA	bhoomika	Bhoomika	4/2/20252:20	31
NAFIDA	nafida	Nafida	4/2/20252:20	31
NETALI	netali	Netali	4/2/20252:20	31
JAYSHREEE	jayshreee	Jayshreee	4/2/20252:20	31

INTRODUCTION TO HTML

HTML: which stands for Hyper Text Markup Language, is the foundation of web pages, providing structure and content by using tags and elements that browsers interpret to display text, images, and other media.

CHARACTERISTICS OF HTML:

1. Markup language: Uses tags to define elements in a document
2. Semantic content: HTML elements that describe the content.
3. Platform- independent: Works on any operating system and device
4. Headings: Organizes content and improves readability
5. Title: Defines the title of the document, which appears in the browser's title bar or in the page's tab
6. Audio: Allows users to incorporate audio files into their webpages
7. Hyper-Link: When a visitor clicks a link on a web page, they are led to another web page or document

BASIC TAGS:

1. <!DOCTYPE html>: Declares the document type as HTML.
2. <html>:The root element of an HTML page, containing all other elements.
3. <head>:Contains meta data about the HTML document, such as the title, character set, and links to external resources.
4. <title>:Defines the title of the HTML page, which is displayed in the browser's title bar or tab.
5. <body>: Contains the visible page content, including text, images, and other elements.
6. <paragraph>OR<p>:Defines a paragraph of text.
7. <script>: redirect from one page to another page.

DESCRIPTION:

1. **HTML**:

   <html>

   ……..

   </html>

2. **HEAD**:

   <head>

   <title> My webpage</title>

   </head>

3. **TITLE**:

   <title>Welcome to my webpage</title>

4. **BODY**:

   <body>

   <h1>Welcome to my webpage</h1>

   </body>

5. HEADING:

   <h1>Main Heading</h1>

   <h2>Sub-Heading</h2>

   <h3>Sub-Sub Heading</h3>

6. **PARAGRAPH**:

   ** ** Visual Studio Code (VS Code) is a powerful, open-source code editor developed by Microsoft. It offers a rich set of features that support development in multiple programming languages, making it a favorite among developers worldwide. With its lightweight design, robust performance, and extensive extension market place, VS Code helps stream line coding workflows efficiently. ** **

7. **LINEBREAKAND HORIZONTAL TAGS:**

**
Line Break</br>**

<hr>Horizontal</hr>

8. LIST:

   a. Order List (ol)

   b. Unordered List (ul)

3.5. Create a Program using html, head, title, body, heading, paragraph, line break, horizontal and list tags.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Visual Studio Code</title>
</head>
<body>
   <h1><center>Visual Studio Code</center></h1>
   <center><imgsrc="C:\Users\spc\Documents\Sebastian_SM220022\image-40.png"
alt="Visual Studio Code Logo" width="300"></center>
   <center><h2>The best platform to manage and run codes.</h2></center>
    <h3>Supported Languages:</h3>
   <ol>
      <li>HTML</li>
```

**
JavaScript
**

**
Python
**

**
**

<h3>Features:</h3>

<ul>

**
Intelligent code completion(IntelliSense)
**

**
Built-in Git support
**

**
Customizable themes and extensions
**

**
**

** Visual Studio Code(VS Code)is a powerful, open-source code editor developed by Microsoft.
**

**It offers a rich set of features that support development in multiple programming languages,
 making it a favorite among developers worldwide. With its lightweight design, robust performance, and extensive extension market place, VS Code helps streamline coding workflows efficiently. **

</body>

</html>

OUTPUT:



3.6Create a Registration page using CSS and HTML.

```
<!DOCTYPE html>

<html>

<head>

<metaname="view point"content="width=device-width, initial-scale=1">

<style>
body{
font- family: Arial, Helvetica, sans-
serif; background-color: black;
}

.{

box-sizing: border-box;

}
```

```
/*Addpadding to containers*/
.container{ padding:16px; background-color: white;
}
/*Full-widthin put
fields*/ input[type=text],
input[type=password]{
width:100%
padding:15px;margin:5
px 0 22px 0;
display:inline-block;
border:none;
background:#f1f1f1;
}
input[type=text]:focus,
input[type=password]:focu
s{ background-color:#ddd;
outline:none;
}
/*Over write default styles of
hr*/ hr{
border:1pxsolid#f1f1f1
; margin-bottom:25px;
}
/*Set a style for the submit button*/
.registerbtn{
background-color:#04AA6D;
color:white;
padding:16px20px;
margin:8px 0;
border:none;
```

```
cursor:pointer,
width:100%
opacity:0.9;
}

.registerbtn:hover{
opacity:1;
}

/*Addabluetextcolortolinks*/
a{
color:dodgerblue;

}

/*Setagrey background color and center the text of the"signin"section*/

.signin{

background-
color:#f1f1f1;          text-
align:center;
}

</style>

</head>

<body>

<formaction="/action_page.php">

<divclass="container">

<h1>Register</h1>

<p>Please fill in this form to create an account.</p>

<hr>

<labelfor="email"><b>Email</b></label>
```

```
<inputtype="text"placeholder="EnterEmail"name="email"id="email"required><br>

<labelfor="psw"><b>Password</b></label>

<inputtype="password"placeholder="Repeat Password"name="psw"id="psw"required><br>

<labelfor="psw-repeat"><b>RepeatPassword</b></label>

<input type="password" placeholder="Repeat Password" name="psw-repeat"
        id="psw- repeat"required><br>

<hr>

<p>Bycreatinganaccountyouagreeto our<ahref="#">Terms&Privacy</a>.</p>

<buttontype="submit"class="registerbtn">Register</button>

</div>

<divclass="containersignin">

<p>Alreadyhaveanaccount?<ahref="#">Sign in</a>.</p>

</div>

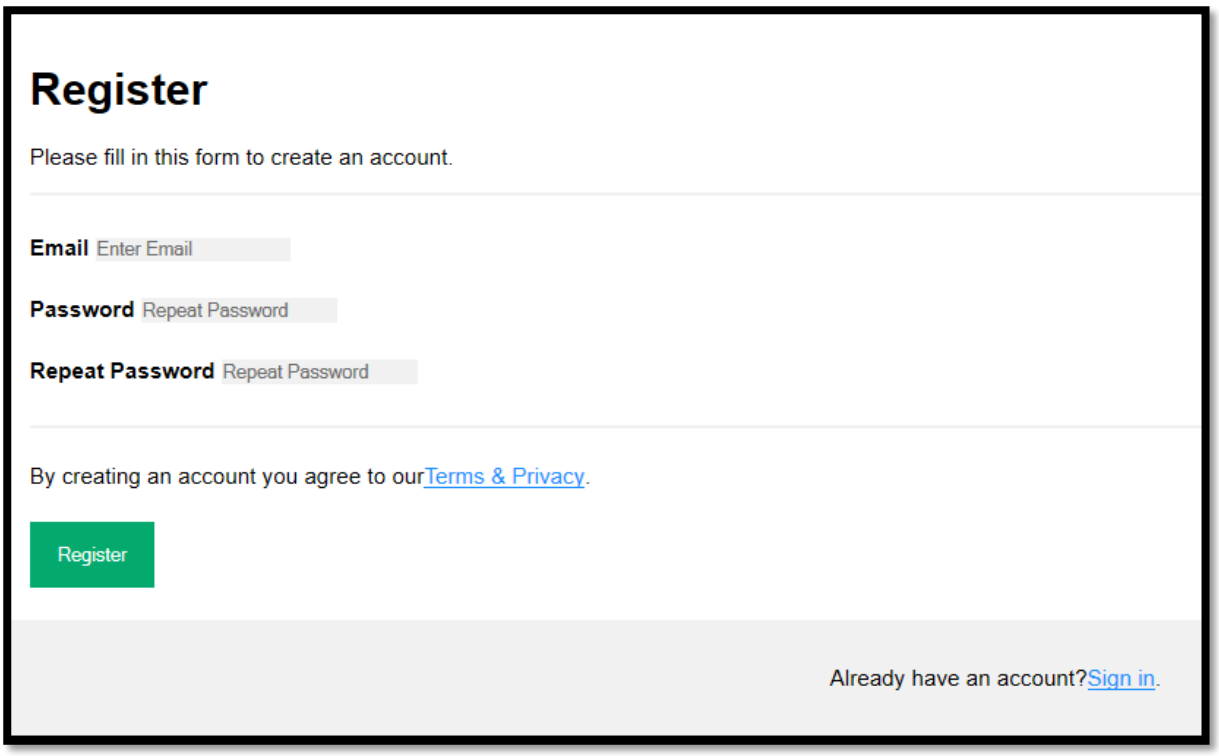
</form>

<ahref="C:\Users\sppc\Desktop\ff.html">clickme</a>

</body>

</html>
```

OUTPUT:


Register
Please fill in this form to create an account.
Email Enter Email
Password Repeat Password
Repeat Password Repeat Password
By creating an account you agree to ourTerms & Privacy.
Register
Already have an account?Sign in.

3.7Create a login page using HTML.

```
<!DOCTYPE html>

<html>

<head>

<title>Login Form</title>

<linkrel="stylesheet
"href="style.css"/>
</head>

<body>

<h2>Login Page</h2><br>

<divclass="Login">

<formid="login"method="g
et"action="login.php">
<label><b>Enter Your Name</
 b></label>

<input
type="text"name="Uname"id="Una
me"placeholder="EnterYourName"
>
<br></br>

<label><b>Password</b></label>

 <input
type="Password"name="pass"id="
pass" placeholder="password">
<br></br>
```

```
<inputtype="button"name="l
og"  id="lod"value="Log  in
Here">
<br></br>

<inputtype="checkbox
"id="chekbox">
<span>Remember me</span>

<br></br>

<ahref="#">Forgetpassword</a>

</form>

</div>

</body>

</html>
```

OUTPUT:



4. DEFINITION DATA WAREHOUSE:

- Data Warehouse is an information storage system for historical data that can be analyzed in numerous ways.
- A data warehouse refers to a database that is maintained separately from organisations operational database.
- Data warehouse allows for the integration of variety of application system. They support information processing by providing a solid platform for consolidated, historical data for analysis.
- It collects the data from multiple heterogeneous database files (flat, text and etc).
- It stores the 5 to 10 years of huge amount of data.
-

CHARACTERISTICS OF DATA WAREHOUSE:

1. SUBJECT ORIENTED

A data warehouse is subject-oriented, as it provides information on a topic rather than the ongoing operations of organizations. Such issues may be inventory, promotion, storage, etc. Never does a data warehouse concentration the current processes. Instead, it emphasized modeling and analyzing decision-making data. It also provides simple and succinct description of the particular subject by excluding details that would not be useful in helping the decision process.

2. INTEGRATED

Integration in Data Warehouse means establishing a standard unit of measurement from the different databases for all the similar data. The data must also get stored in a simple and universally acceptable manner within the Data Warehouse. Through combining data from various sources such as a mainframe, relational databases, flat files, etc., a data warehouse is created. It must also keep the naming conventions, format, and coding consistent .Such an application assists in robust data analysis.

Consistency must be maintained in naming conventions, measurements of characteristics, specification of encoding, etc.

3. **TIME-VARIANT**

Compared to operating systems, the time horizon for the data warehouse is quite extensive. The data collected in a data warehouse is acknowledged over a given period and provides historical information. It contains a temporal element, either explicitly or implicitly.

One such location in there cord key system where Data Warehouse data shows time variations. Each primary key contained with the DW should have an element of time either implicitly or explicitly. Just like the day, the month of the week, etc.

4. **NON-VOLATILE**

The data warehouse is non-volatile, meaning that prior data will not be erased when new data are entered into it. Data is read-only, only updated regularly. It also assists in analyzing historical data and in understanding what and when it happened. The transaction process, recovery, and competitiveness control mechanisms are not required. In the Data Warehouse environment, activities such as deleting, updating, and inserting that are performed in an operational application environment are omitted.

DATAWAREHOUSECOMPONENTS:

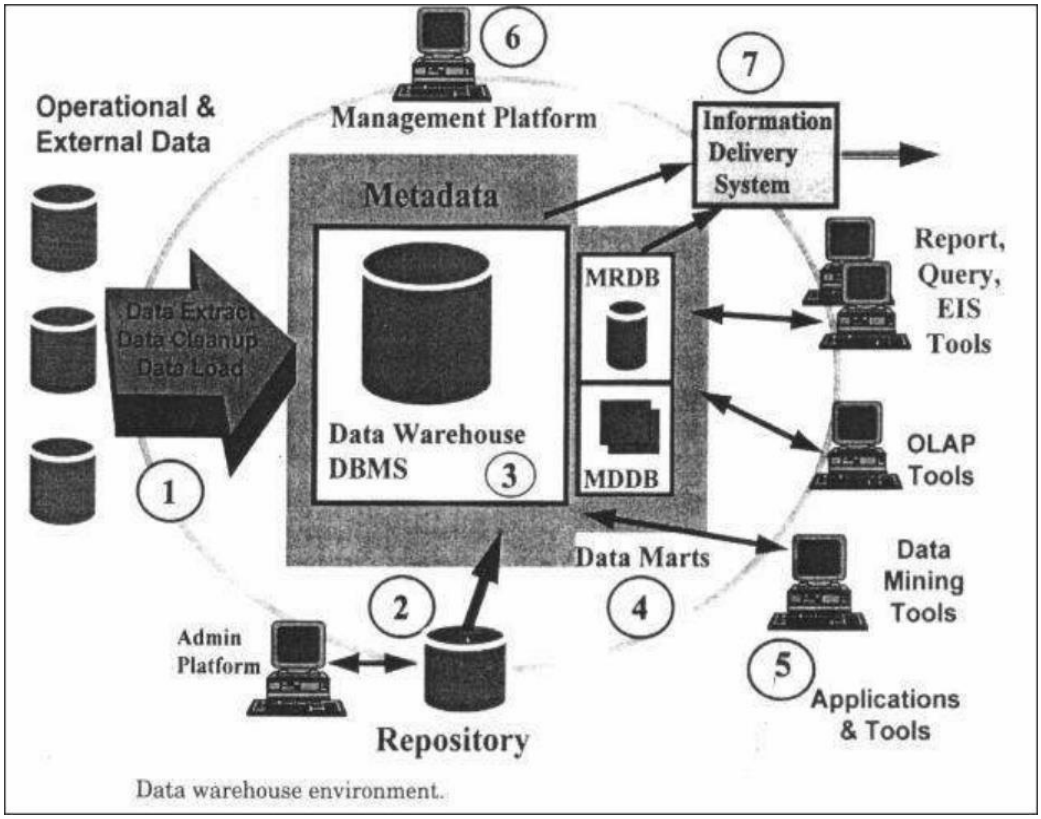
- Thedatawarehousearchitectureisbasedonthedatabasemanagementsystemserver.
- Thecentralinformationrepositoryissurroundedbynumberofkeycomponents
- Data warehouse is an environment, not a product which is based on relational database management system that functions as the central repository for informational data.
- The data entered into the data warehouse transformed into an integrated structure and format.

The transformation process involves conversion, summarization, filtering and condensation.

- The data warehouse must be capable of holding and managing large volumes of data as well as different structure of data structures over the time.

KEY COMPONENTS

1. Data sourcing, clean-up, transformation, and migration tools.
2. Meta data repository
3. Warehouse/database technology
4. Data marts
5. Data query, reporting, analysis, and mining tools
6. Data warehouse administration and management



Data warehouse environment.

1. DATA SOURCING, CLEAN-UP, TRANSFORMATION, AND MIGRATION TOOLS

- They perform conversions, summarization, key changes, and structural changes.
- Data transformation is required to be used by decision support tools.
- The transformation produces programs and control statements.
- It moves the data into the data warehouse from multiple operational systems.

FUNCTIONALITIES OF THESE TOOLS:

- To remove unwanted data from operational databases.
- Converting to common data names and attributes.
- Calculating summaries and derived data.
- Establishing defaults for missing data.
- Accommodating source data definition changes

2. META DATA REPOSITORY

1. TECHNICAL META DATA:

It contains information about data warehouse data used by warehouse designers and administrators to carry out development and management tasks. It includes:

- Information about data stores.
- Transformation descriptions, i.e., mapping methods from operational databases to warehouse databases.
- Data mapping operations.
- Access authorization, backup history, archive history, info delivery history, data acquisition history, data access, etc.

2. BUSINESS META DATA:

It contains information that helps users understand the data stored in the data warehouse. It includes:

- Subject areas and info object types, including queries, reports, images, videos, audio clips, etc.
- Internet homepages.
- Information related to the info delivery system.

Metadata helps users to understand content and find the data. Metadata is stored in a separate data store known as an informational directory or metadata repository, which helps to integrate, maintain, and view the contents of the data wareho

CHARACTERISTICS OF AN INFO DIRECTORY/METADATA:

- It is the gateway to the data warehouse environment.
- It supports easy distribution and replication of content for high performance and availability.
- It should be searchable by business-oriented keywords.
- It should act as a launch platform for end-users to access data and analysis tools.
- It should support the sharing of information.
- It should support scheduling options for requests.
- It should support and provide interfaces to other applications.
- It should support end-user monitoring of the status of the data warehouse environment

3. WAREHOUSE/DATABASE TECHNOLOGY

The data warehouse database is the central part of the data warehousing environment. It is implemented based on RDBMS (Relational Database Management System) technology.

4. DATA MARTS

A data mart is a subset or an aggregation of the data stored in a primary data warehouse. It includes a set of information pieces relevant to a specific business area, corporate department, or category of users.

Reasons for Using Data Marts:

- Extremely urgent user requirements.
- The absence of a budget for a full-scale data warehouse strategy.
- The decentralization of business need

5. DATA QUERY, REPORTING, ANALYSIS, AND MINING TOOLS

The purpose of these tools is to provide information to business users for decision-making. There are five categories:

- Data query and reporting tools
- Application development tools
- Executive Information System(EIS)tools
- OLAP(Online Analytical Processing) tools
- Data mining tools

6. DATA WAREHOUSE ADMINISTRATION AND MANAGEMENT

The management of a data warehouse includes:

- Security and priority management
- Monitoring updates from multiple sources
- Data quality checks
- Managing and updating meta data
- Auditing and reporting data warehouse usage and status
- Purging data
- Replicating, sub setting, and distributing data
- Back-up and recovery
- Data warehouses to rage management, including capacity planning, hierarchical storage management, and purging of aged da

7. INFORMATION DELIVERY SYSTEM

- It is used to enable the process of subscribing to data warehouse information.
- Delivery to one or more destinations according to a specified scheduling algorithm.

Building a Data Warehouse

REASONS TO CONSIDER DATA WAREHOUSING:

There are two factors that drive organizations to build and use data warehouses: Business Factors:

- Business users want to make decisions quickly and correctly using all available data.

Technological Factors:

- To address the incompatibility of operational data stores.
- IT infrastructure is changing rapidly, with increasing capacity and decreasing costs, making it easier to build a data warehouse.

APPROACHES TO BUILDING A DATA WAREHOUSE:

1. **TOP-DOWN APPROACH:**
    - Starts with the overall design and planning.
    - Useful when the technology is mature and well-known, and business problems are clear and well-understood.
2. **BOTTOM-UP APPROACH:**
    - Starts with experiments and proto types.
    - Useful in the early stages of business modeling and technology development.
    - Allows an organization to move forward at considerably less expense and evaluate the benefits before making significant commitments.
3. **COMBINATION OF BOTH APPROACHES:**
    - Exploits the strategic nature of the top-down approach while retaining the rapid implementation of the bottom-up approach.

DATA WAREHOUSE DESIGN PROCESS:

1. **CHOOSE A BUSINESS PROCESS TOMODEL:**
    - Example: Orders, invoices, shipments, inventory, account administration, sales, general ledger, etc.
    - If the process is organizational and involves multiple complex object collections, a data warehouse model should be followed.
    - If the process is department a land focuses on analyzing one kind of business process, a data mart model should be chosen.
2. **CHOOSE THE GRAIN OF THE BUSINESS PROCESS:**
    - The grain is the fundamental, atomic level of data to be represented in the fact table, e.g., individual transactions, daily snapshots, etc.
3. **CHOOSE THE DIMENSIONS FOR EACH FACT TABLE RECORD:**
    - Typical dimensions include: Time, Item, Customer, Supplier, Warehouse, Transaction Type, and Status.
4. **Choose the Measures to Populate Each Fact Table Record:**

WHAT IS RELATIONSHIP IN DBMS?

A relationship in DBMS is association between two or more entities. Entities are the real world objects which hold data, participate in these relationships. Connection between different entities are represented by a diamond shapes in ER (entity relationship) diagrams. Relationships enables the separation and organization of data across multiple tables which facilitate the data management and data retrieval.

Primary Terminologies

- Entity: Entity is a real world instance or object that can store data. Each entity is represented by a table in the database.
- Relationships: It is relation or links between two or more entities. Relationships define how data is related and structured across multiple tables in a database.
- ER (Entity-Relationship) Diagram: It is a graphical representation of entities and their relationship to each other. In ER diagrams, entities are represented by rectangles and the relation between them is represented by diamond.

TYPES OF RELATIONSHIPS IN DBMS

There are three primary types of relationships in DBMS. Each type of relationship plays a unique role in database design.

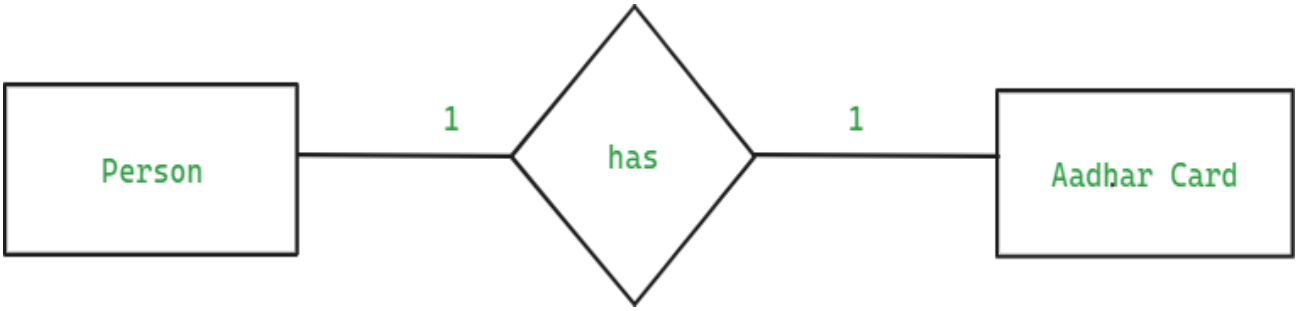
- One-to-One relationship
- One-to-Many or Many-to-One relationship
- Many-to-Many relationship

One-to-One Relationship

In one-to-one relationship a single record in one table is related with a single record in other table and vice versa. These types of relationships are relatively rare and commonly used for security or organizational reasons.

Example:

Consider two entities "Person" and "Aadhar card". Each person can have only one Aadhar card and each Aadhar card is assigned to only one person.



Implementation:

Here we will see how to implement one-to-one relationship using SQL.

```
CREATE TABLE Person (
  PersonID INT PRIMARY
  KEY, Name
  VARCHAR(100) );

CREATE TABLE AadharCard (
  AadharCardID INT PRIMARY
  KEY, PersonID INT UNIQUE,
  IssueDateDATE,
  FOREIGNKEY(PersonID)REFERENCESPerson(PersonID)
);

--Inserting sample data
INSERTINTOPerson(PersonID,Name)VALUES(1,'JohnDoe');
INSERT INTO AadharCard (AadharCardID, PersonID, IssueDate) VALUES
(1001, 1, '2022-01-01');

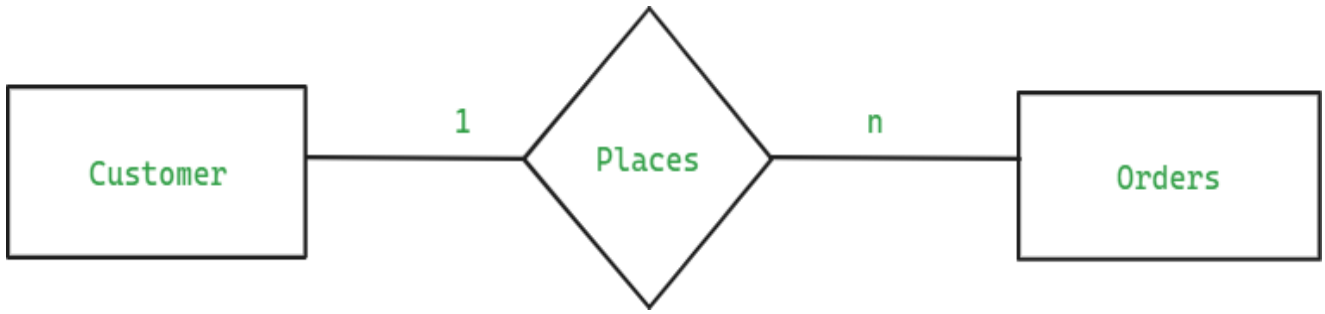
--Query to retrieve data
```

Output Table:

NAME	AADHARCARDID	ISSUEDATE
John Doe	1001	01-JAN-22

One-to-Many or Many-to-One Relationship

In one-to-many or many-to-one relationship, a single record in one table can be associated with multiple records in another table and this is the most common type of relationship in DBMS.



Consider two entities "customer" and "order". Each customer can place multiple orders but each order is placed by only one customer.

Implementation:Here we will see how to implement one-to-many or many-to-one relationship using SQL.

```
CREATE TABLE Customer (
  CustomerID INT PRIMARY
  KEY, Name VARCHAR(100)
);

CREATE TABLE Orders (
  OrderID INT PRIMARY
  KEY,
  OrderDate DATE,
  CustomerID INT,
```

```
INSERT INTO Customer (CustomerID, Name) VALUES (1,
'Alice'); INSERT INTO Orders (OrderID, OrderDate,
CustomerID) VALUES (101, TO_DATE('2022-01-10',
'YYYY-MM-DD'), 1);
INSERT INTO Orders (OrderID, OrderDate, CustomerID)
VALUES (102, TO_DATE('2022-01-15', 'YYYY-MM-DD'),
1);

--Querytoretrievedata
SELECT c.Name, o.OrderID,
o.OrderDate FROM Customer c
JOINOrdersoONc.CustomerID=o.CustomerID;
```

OutputTable:

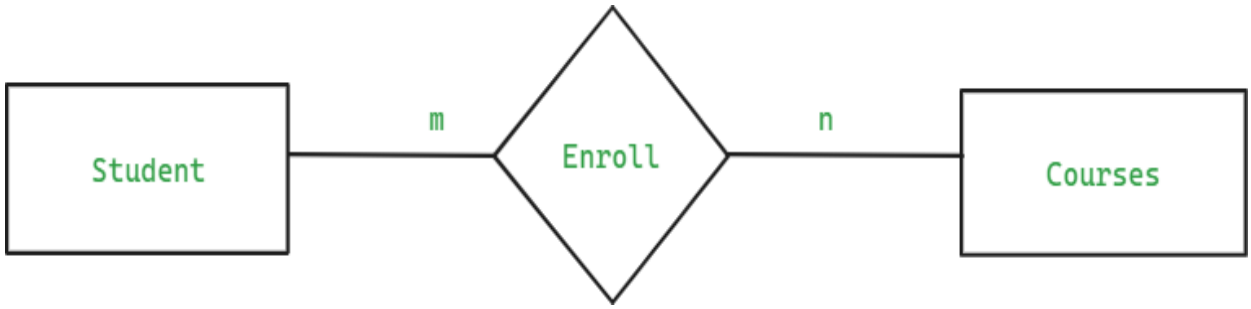
NAME	ORDERID	ORDERDATE
Alice	101	10-JAN-22
Alice	102	15-JAN-22

Many-to-Many Relationship

A many-to-many relationship is relationship in which one multiple records in one table are associated with multiple records in another table. This relationship is mainly implemented using junction table.

Example:

Consider two entities "Student" and "Course" where each student can enroll in multiplecourses and each course can have multiple students enrolled in it



8. Implementation:

9. Here we will see how to implement many-to-many relationship using SQL. CREATE TABLE Student (Key, Name VArchhar(100)

CREATE TABLE Course (

CourseID INT PRIMARY KEY,

10. CourseNameVARCHAR(100)

11.);

12.

13.

14. CREATE TABLE Enrollment (StudentID INT,

15. CourseID INT, EnrollmentDate DATE,

16. PRIMARYKEY(StudentID,CourseID),

17. FOREIGN KEY (StudentID) REFERENCES Student(StudentID), FOREIGN KEY (CourseID) REFERENCES

```
    Course(CourseID)
);

INSERT INTO Student (StudentID, Name) VALUES (1, 'Bob'); INSERT INTO Student (StudentID, Name) VALUES (2, 'Charlie');

INSERT INTO Course (CourseID, CourseName) VALUES (101, 'Math'); INSERT INTO Course (CourseID, CourseName)VALUES (102,'Science');

INSERT INTO Enrollment (StudentID, CourseID, EnrollmentDate) VALUES (1, 101, TO_DATE('2022-01-10', 'YYYY-MM-DD'));
INSERT INTO Enrollment (StudentID, CourseID, EnrollmentDate) VALUES (1, 102, TO_DATE('2022-01-12', 'YYYY-MM-DD'));
KEY,                Name
VARCHAR(100)
```

OutputTable:

STUDENTNAME	COURSENAME	ENROLLMENTDATE
Bob	Math	10-JAN-22
Charlie	Math	15-JAN-22
Bob	Science	12-JAN-22

These are basic types of relationship but beyond these relationships there are advanced relationship concepts that further enhances database design:

Self-Referencing Relationships

A self-referencing relationship is also known as recursive relationship and itis useful is cases when a table has relationship with itself. It is used for representing hierarchical data.

Example:

An "Employee" entity where each employee can have manager who is also an employee.

Implementation:

Here we will see how to implement self-referencing relationship using SQL.

```
CREATE TABLE Employee (
  EmployeeID INT PRIMARY
  KEY, Name VARCHAR(100),
  ManagerIDINT,
  FOREIGNKEY(ManagerID)REFERENCESEmployee(EmployeeID)
);

--Inserting sample data
INSERT INTO Employee (EmployeeID, Name, ManagerID) VALUES (1,
'CEO', NULL); INSERT INTO Employee (EmployeeID, Name, ManagerID)
VALUES (2, 'Manager', 1); INSERT INTO
Employee(EmployeeID,Name,ManagerID)VALUES (3,'Worker',2);

--Query to retrieve data
SELECT e1.Name AS EmployeeName, e2.Name AS
```

Output Table:

;

EMPLOYEENAME	MANAGERNAME
C	-
Manager	CEO
Worker	Manager

ER-MODEL-KEY CONSTRAINTS

Constraints are used for modeling limitations on the relations between entities.

There are two types of constraints on the Entity Relationship (ER) model −

- Mapping cardinality or cardinality ratio.
- Participation constraints.

Mapping Cardinality

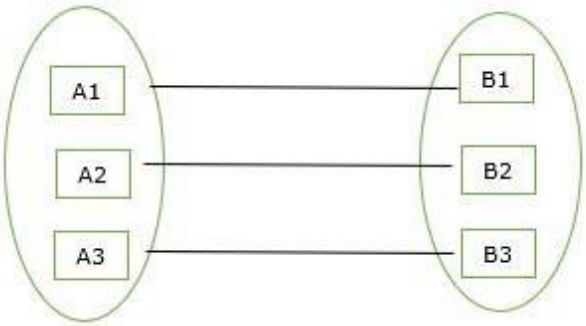
It is expressed as the number of entities to which another entity can be associated via a relationship set.

For the binary relationship set there are entity set A and B then the mapping cardinality can be one of the following −

- One-to-one
- One-to-many
- Many-to-one
- Many-to-many

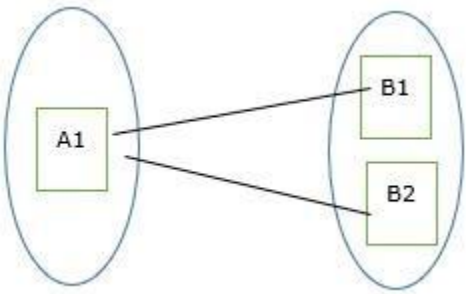
One-to-one relationship

An entity set A is associated with at most one entity in B and an entity in B is associated with at most one entity in A.



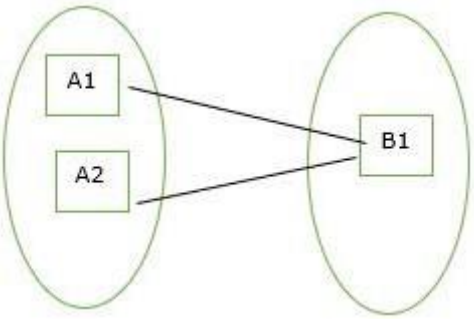
One-to-many relationship

An entity set A is associated with any number of entities in B with a possibility of zero and an entity in B is associated with at most one entity in A.



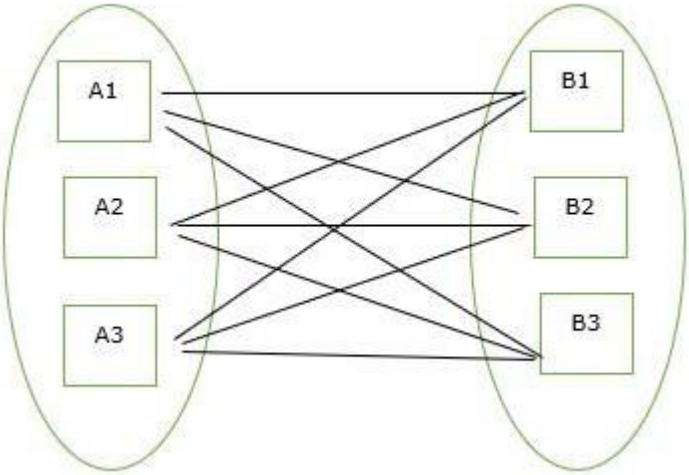
Many-to-one relationship

An entity set A is associated with at most one entity in B and an entity set in B can be associated with any number of entities in A with a possibility of zero.



Many-to-many relationship

An entity set A is associated with any number of entities in B with a possibility of zero and an entity in B is associated with any number of entities in A with a possibility of zero.

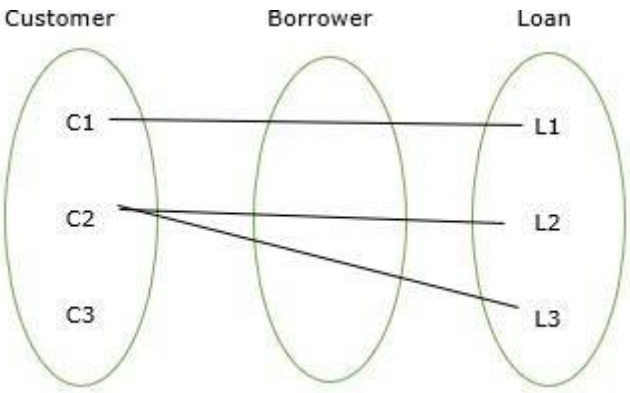


Participation Constraints

Participate constraints are two types as mentioned below−

- Total participation
- Partial participation

The participation constraints are explained in the diagram below−



Here, the customer to Loan is partial participation and the loan to the customer is total participation.

Total participation

The participation of an entity set E in a relationship set R is said to be total if every entity in E Participates in at least one relationship in R.

For Example−Participation of loan in the relationship borrower is total participation.

Partial Participation

If only some of the entities in E participate in relationship R, then the participation of E in R is said to be partial participation.

For example−Participation of customers in the relationship borrower is partial participation.

What is Participation Constraints?

Participation Constraints in database management refer to rules that determine the minimum and maximum participation of entities or relationships in a given relationship set

In an Entity-Relationship(ER) model, key constraints refer to the rules that specify how entities are uniquely identified and how relationships between them are defined. Key constraints are important to ensure the integrity of the database structure. Here are the key constraints commonly used in ER models**:**

1. Primary Key Constraint:

- Definition: The primary key uniquely identifies each entity in an entity set.
    - No two entities in the entity set can have the same value for the primary key.
    - The primary key cannot contain NULL values.
    - It is used to enforce uniqueness within a particular entity.
    - In ER diagrams, a primary key is often underlined.

Example: In an entity Employee, the Employee_ID attribute might be chosen as the primary key because each employee has a unique ID.

2. Foreign Key Constraint:

- Definition: A foreign key is an attribute (or a set of attributes) that creates a relationship between two entities.
    - A foreign key in one entity set refers to the primary key of another entity set.
    - Foreign keys can have NULL values unless explicitly restricted by the relational schema.
    - This constraint helps to maintain referential integrity by ensuring that relationships between entities are valid.

Example: In an entity Department, the attribute Manager_ID might be a foreign key that refers to Employee_ID in the Employee entity.

3. **Candidate Key Constraint:**

- Definition: A candidate key is an attribute (or a set of attributes) that could uniquely identify each entity in an entity set, but there may be multiple candidate keys.
    - Every candidate key is a potential primary key.
    - The entity set can have multiple candidate keys, but only one will be chosen as the primary key.
    - A candidate key must be unique and non-null.

Example: In an entity Student, both Student_ID and Email could be candidate keys because each of these attributes can uniquely identify a student.

4. **Super key Constraint:**

- Definition: A super key is a set of one or more attributes that can uniquely identify each entity in an entity set.
    - A super key can contain extra attributes beyond those required for uniqueness.
    - Every primary key is a super key, but not every super key is a primary key.

Example: In the Employee entity, the combination of Employee_ID and Department_ID can be a super key, even though Employee_ID alone is sufficient to uniquely identify an employee.

5. **Weak Entity and Identifying Relationship:**

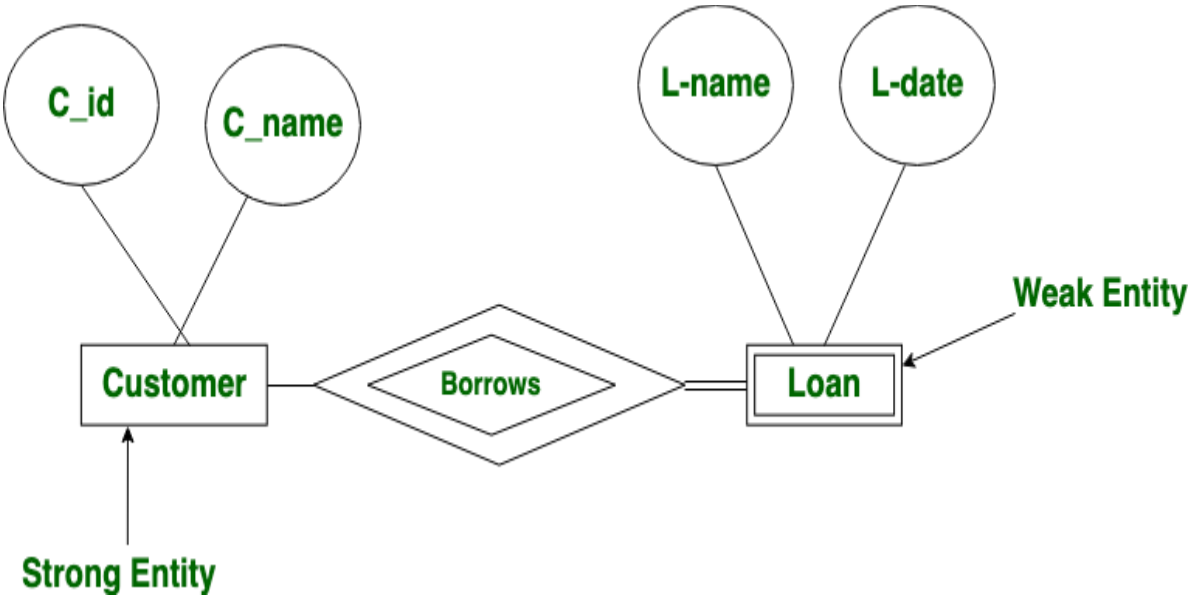
- Definition: A weak entity is an entity that cannot be uniquely identified by its own attributes alone and requires the help of another entity (usually its owner or parent entity) to form a unique identifier.
    - The weak entity does not have a primary key on its own but relies on the primary key of another entity.
    - An identifying relationship connects a weak entity to its owner entity, which helps form a composite key for the weak entity.

Example: An entity Order Item might be a weak entity that depends on the Order entity for its identification, with the primary key being a composite of Order_ID and Item_ID.

Weak Entity: An entity that cannot be uniquely identified by its own attributes and relies on the relationship with other entity is called weak entity. The weak entity is represented by a double rectangle.

What is a Weak Entity?

A weak entity is dependent on a strong entity to ensure its existence. Unlike a strong entity, a weak entity does not have any primary key. It instead has a partial discriminator key. A weakentityis represented by a double rectangle. The relation between one strong and one weak entity is represented by a double diamond. This relationship is also known as an identifying relationship**.**



In ER models, strong entities can exist independently, whereas weak entities depend on strong entiti

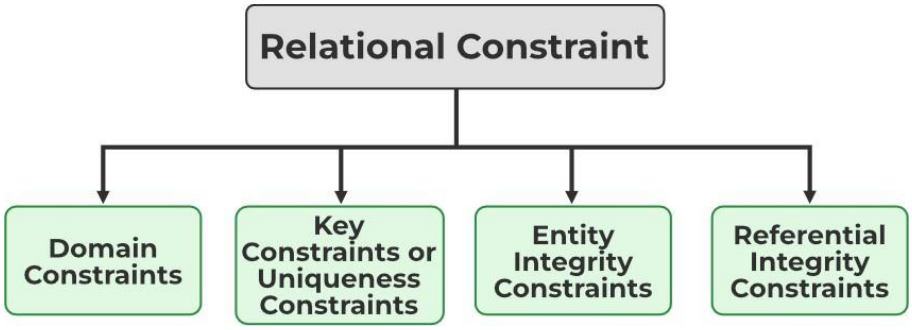
Differences between Strong Entity and Weak Entity:

Strong Entity	Weak Entity
Strong entity always has a primary key.	While a weak entity has a partial discriminator key.
Strong entity is not dependent on any other entity.	Weak entity depends on strong entity.
Strong entity is represented by a single rectangle.	Weak entity is represented by a double rectangle.
Two strong entity's relationship is represented by a single diamond.	While the relation between one strong and one weak entity is represented by a double diamond.
Strong entities have either total participation or partial participation.	A weak entity has a total participation constraint.

(Unit2) Relational Constraints

These are the restrictions or sets of rules imposed on the database contents. It validates the quality of the database. It validates the various operations like data insertion, updating, and other processes that have to be performed without affecting the integrity of the data. It protects us against threats/damages to the database. Mainly Constraints on the relational database are of 4 types

- Domain constraints
- Key constraints or Uniqueness Constraints
- Entity Integrity constraints
- Referential integrity constraints



Types of Relational Constraints

1. Domain Constraints

- Every domain must contain atomic values (smallest indivisible units) which means composite and multi-valued attributes are not allowed.

- We perform a datatype check here, which means when we assign a datatype to a column we limit the values that it can contain. Eg. If we assign the datatype of attribute age as int, we can't give it values other than intdatatyp

EID	Name	Phone
01	Bikash Dutta	123456789 234456678

Example:

Explanation: In the above relation, Name is a composite attribute and Phone is a multi-values attribute, so it is violating domain constraint.

2. Key Constraints or Uniqueness Constraints

- These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
- A relation can have multiple keys or candidate keys (minimal super key), out of which we choose one of the keys as the primary key, we don't have any restriction on choosing the primary key out of candidate keys, but it is suggested to go with the candidate keywith less number of attributes.
- Null values are not allowed in the primary key, hence Not Null constraint is also part of the key constraint.

EID	Name	Phone
01	Bikash	6000000009
02	Paul	9000090009
01	Tuhin	9234567892

Explanation: In the above table, EID is the primary key, and the first and the last tuple have the same value in EID i.e., 01, so it is violating the key constraint.

3. Entity Integrity Constraints

- Entity Integrity constraints say that no primary key can take a NULL value, since using the primary key we identify each tuple uniquely in a relation.

Example:

EID	Name	Phone
01	Bikash	9000900099
02	Paul	600000009
NULL	Sony	9234567892

Explanation: In the above relation, EID is made the primary key, and the primary key can't take NULL values but in the third tuple, the primary key is null, so it is violating Entity Integrity constraints.

4. Referential Integrity Constraints

- The Referential integrity constraint is specified between two relations or tables and used to maintain the consistency among the tuples in two relations.

- This constraint is enforced through a foreign key, when an attribute in the foreign key of relation R1 has the same domain(s) as the primary key of relation R2, then the foreign key of R1 is said to reference or refer to the primary key of relation R2.

- The values of the foreign key in a tuple of relation R1 can either take the values of the primary key for some tuple in relation R2, or can take NULL values, but can't be empty.

Example:

EID	Name	DNO
01	Divine	12
02	Dino	22

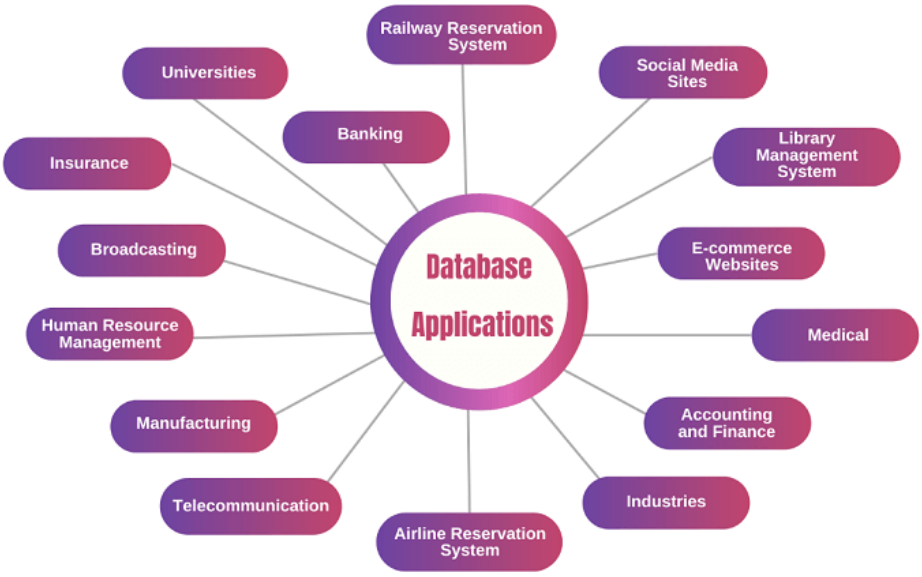
EID	Name	DNO
04	Vivian	14

DNO	Place
12	Jaipur
13	Mumbai
14	Delhi

Explanation: In the above tables, the DNO of Table 1 is the foreign key, and DNO in Table 2 is the primary key. DNO=22inthe foreign key of Table1 is not allowed because DNO=22 is not defined in the primary key of table 2. Therefore, Referential integrity constraints are violated here.

Examples of Database Applications

Nowadays, any business that has small or large amounts of data needs a database to store and manage the information. The database is an easy, reliable, secure, and efficient way to maintain business information. There are many applications where databases are used.



1. **Universities:**

It is an undeniable application of the database. Universities have so much data which can be stored in the database, such as student information, teacher information, non-teaching staff information, course information, section information, grade report information, and many more. University information is kept safe and secure in the database.

Anyone who needs information about the student, teacher, or course can easily retrieve it from the database. Everything needs to be maintained because even after ten years, information may be required, and the information may be useful, so maintaining complete information is the primary responsibility of any university or educational institution.

2. **Banking**:

It is one of the major applications of databases. Banks have a huge amount of data as millions of

people have accounts that need to be maintained properly. The database keeps the record of each user in a systematic manner. Banking databases store a lot of information about account holders. It stores customer details, asset details, banking transactions, balance sheets, credit card and debit card details, loans, fixed deposits, and much more. Everything is maintained with the help of a database.

3. **Railway Reservation System:**

It is an inevitable area of application of databases. They store information such as passenger name, mobile number, booking status, reservation details, train schedule, employee information, account details, seating arrangement, route & alternate route details, etc. All the information needs to be maintained, so railways use a database management system for their efficient storage and retrieval purpose.

4. **Social Media Sites:**

Nowadays, everyone has a smartphone and accounts on various social media sites like Facebook, LinkedIn, Pinterest, Twitter, Instagram, etc. People can chat with their friends and family and make new friends from all over the world. Social media has millions of accounts, which means they have a huge amount of data that needs to be stored and maintained. Social media sites use databases to store information about users, images, videos, chats, etc.

5. **Library Management System:**

There are hundreds and thousands of books in the library, so it is not easy to maintain the records of the books in a register ordinary, so a database management system is used which maintains the information of the library efficiently. The library database stores information like book name, issue date, author name, book availability, book issuer name, book return details, etc

6. **E- commerce Websites:**

E-commerce websites are one of the prominent applications of the database. Websites such as Flipkart, Myntra, Amazon, Nykaa, Snapdeal, Shopify, and many more, are online shopping Websites where people buy items online. These websites use databases to securely store and maintain customer details, product details, dealer details, purchase details, bank & card details, transactions details, invoice details, etc. You can analyze the sales and maintain the inventory with the help of a database

1. **Medical:**

There is a lot of important data collection in the medical field, so it is necessary to use the database to store data related to the medical field, such as patient details, medicine details, practitioner details, surgeon details, appointment details, doctor schedule, patient discharge details, payment detail, invoices, and other medical records. The database management system is a boon for the medical field because it helps doctors to monitor their patients and provide better care.

2. **Accounting and Finance:**

When there is big data regarding accounting and finance, there is a need to maintain a large amount of data, which is done with the help of a database. The database stores data such as accounting details, bank details, purchases of stocks, invoice details, sales records, asset details, etc. Accounting and finance database helps in maintaining and analyzing historical data.

3. **Industries:**

The database management system is the main priority of industries because they need to store huge amounts of data. The industry database stores customer details, sales records, product lists, transactions, etc. All the information is kept secure and maintained by the database.

4. **Airline Reservation System:**

It is one of the applications of database management systems that contain data such as passenger name, passenger check-in, passenger departure, flight schedule, number of flights, distance from source to destination, reservation information, pilot details, accounting detail, route detail, etc. The database provides maintenance and security to airline data**.**

5. **Telecommunication:**

We cannot deny that telecommunication has brought a remarkable revolution worldwide. The Telecom field has huge data, and it is very difficult to manage big data without a database; that is why a telecom database is required, which stores data such as customer names, phone numbers, calling details, prepaid & post-paid connection records, network usage, bill details, balance details

1. **Manufacturing:**

In the manufacturing field, a lot of data needs to be maintained regarding supply chain management, so the database maintains the data such as product details, customer information, order details, purchase details, payment info, worker's details, invoice, etc. Manufacturing companies produce and supply products every day, so it is important to use a database.

2. Human Resource Management:

Any organization will definitely have employees, and if there are a large number of employees, then it becomes essential to store data in a database as it maintains and securely saves the data, which can be retrieved and accessed when required. The human resource database stores data such as employee name, joining details, designation, salary details, tax information, benefits & goodies details, etc.

3. Broadcasting:

Broadcasting is distributing video and audio content to a dispersed audience by television, radio, or other means. Broadcasting database stores data such as subscriber information, event recordings, event schedules, etc., so it becomes important to store broadcasting data in the database.

4. Insurance:

An insurance company needs a database to store large amounts of data. Insurance database stores data such as policy details, user details, buyer details, payment details, nominee details, address details, etc.

Data Abstraction and Data Independence

Database systems comprise complex data structures. In order to make the system efficient in terms of retrieval of data, and reduce complexity in terms of usability of users, developers use abstraction i.e. hide irrelevant details from the users. This approach simplifies database design.

Level of Abstraction in a DBMS

There are mainly 3 levels of data abstraction:

- Physical or Internal Level
- Logical or Conceptual Level
- View or External Level

Physical or Internal Level

This is the lowest level of data abstraction. It tells us how the data is actually stored in memory. Access methods like sequential or random access and file organization methods like B+ trees and hashing are used for the same. Usability, size of memory, and the number of times the records are factors that we need to know while designing the database.

Suppose we need to store the details of an employee. Blocks of storage and the amount of memory used for these purposes are kept hidden from the user.

Logical or Conceptual Level

This level comprises the information that is actually stored in the database in the form of tables. It also stores the relationship among the data entities in relatively simple structures. At this level, the information available to the user at the view level is unknown.

We can store the various attributes of an employee and relationships, e.g. with the manager can also be stored.

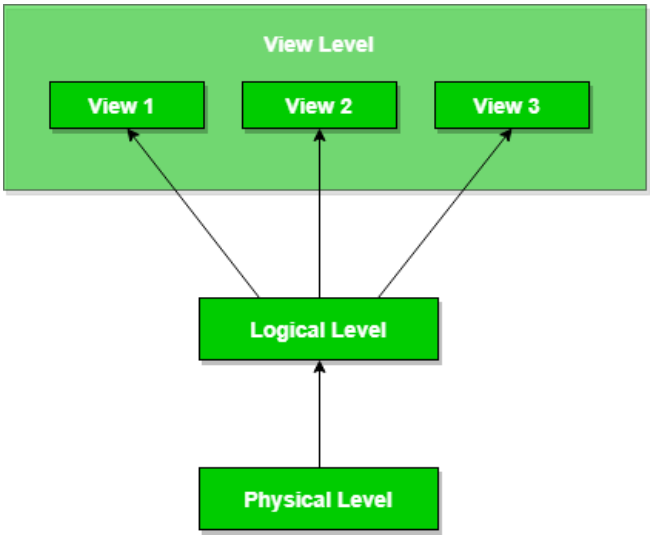
The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as physical data independence. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction

View or External Level

This is the highest level of abstraction. Only a part of the actual database is viewed by the users. This level exists to ease the accessibility of the database by an individual user. Users view data in the form of rows and columns. Tables and relations are used to store data. Multiple views of the same database may exist. Users can just view the data and interact with the database, storage and implementation details are hidden from them. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system

Example: In case of storing customer data,

- Physical level– it will contains block of storages (bytes, GB, TB, etc)
- Logical level– it will contain the field sand the attributes of data.
- View level– it works with CLIor GUIaccess of database



Data Abstraction

The main purpose of data abstraction is to achieve data independence in order to save the time and cost required when the database is modified or altered.

Data Independence

Data Independence is mainly defined as a property of DBMS that helps you to change the database schema at one level of a system without requiring to change the schema at the next level. It helps to keep the data separated from all program that makes use of it. We have namely two levels of data independence arising from these levels of abstraction:

- Physical level data independence
- Logical level data independence

It refers to the characteristic of being able to modify the physical schema without any alterations to the conceptual or logical schema, done for optimization purposes, e.g., the Conceptual structure of the database would not be affected by any change in storage size of the database system server. Changing from sequential to random access files is one such example. These alterations or modifications to the physical structure may include:

- Utilizing new storage devices.
- Modifying data structures used for storage.
- Altering indexes or using alternative file organization techniques etc.

Logical Level Data Independence

It refers characteristic of being able to modify the logical schema without affecting the external schema or application program. The user view of the data would not be affected by any changes to the conceptual view of the data. These changes may include insertion or deletion of attributes, altering table structures entities or relationships to the logical schema, etc.

OPERATING SYSTEM(OS)

Definition. An operating system (OS) is the low-level software layer that sits between hardware and everything else you run. It boots when the machine powers on, takes control of the CPU, memory, storage and devices, and provides a consistent environment for applications and users.

Goals

- **Abstraction**: hide messy hardware details behind clean interfaces (files instead of disk blocks, sockets instead of raw NIC registers).
- **Resourcemanagement**: allocate CPU time, RAM, disk, and I/O fairly and efficiently.
- **Protection&security**: isolate processes, enforce permissions, prevent one program from crashing the whole system.
- **Convenience**: offer user interfaces (CLI/GUI), system utilities, and APIs.

Main components.

- **Kernel** – the privileged core. Monolithic kernels (Linux, Unix) pack most services inside; microkernels (Minix, seL4) keep the kernel tiny and run drivers/servers in user space; hybrid kernels (Windows, macOS XNU) mix both.
- **Process & thread manager**: creates processes, schedules them on cores (preemptive multitasking, priorities, time slices), handles context switches, synchronization primitives (locks, semaphores)
- **Memory manager**: virtual memory, paging/segmentation, page tables, swap space, protection bits; gives each process its own address space.
- **File system**: organizes storage into files/directories, metadata, permissions; examples—NTFS, ext4, APFS, FAT32.
- **Device drivers & I/O subsystem**: drivers speak the hardware language; the OS provides buffering, caching, interrupts/DMA.
- **System calls & APIs**: the boundary where user programs request services (e.g., read(), fork(), open()). Libraries (glibc, Win32) wrap these calls.
- **User interface**: shell/CLI and/or graphical shell (Windows Explorer, GNOME, macOS Finder).

Major families & examples.

- **Desktop/Server**: Windows (NT kernel), macOS (XNU hybrid, Mach + BSD), Linux distributions (Ubuntu, Fedora, Debian—same kernel, different userlands). UNIX heritage underlies macOS, Linux, BSD, Solaris.
- **Mobile/Embedded**: Android (Linux kernel + Android runtime), iOS/iPadOS (XNU-based), RTOSes like FreeRTOS, VxWorks for real-time constraints.
- **Special-purpose**: hypervisors (VMware ESXi, Xen) that act as thin OSes to run other OSes; mainframe OSes (z/OS) for throughput & reliability.

How an OS boots (high-level).

- . Firmware (BIOS/UEFI) initializes hardware, finds a bootloader.
- Bootloader loads the kernel into memory and hands control over.
- Kernel starts init/systemd (PID 1), mounts root filesystem, launches services and user session.

Concurrency & protection.

- Scheduling: time-sharing (round-robin, CFS in Linux), priority classes, real-time policies.
- Synchronization: prevent race conditions via mutexes, semaphores, condition variables.
- Isolation: hardware rings (kernel mode vs user mode), memory protection units, system-call gate.

Storage & persistence.

File systems handle block allocation, journaling (to survive crashes), caching (page cache), access control lists. Modern OSes add encryption (BitLocker, FileVault, LUKS), snapshots (APFS, Btrfs).

Security & reliability mechanisms.

- User accounts, privileges, capabilities; sandboxing/containers; ASLR, DEP, secure boot; auditing and logging.
- Fault tolerance via watchdog timers, recovery modes, journaling.

Trends.

- Virtualization & containers: hypervisors and container runtimes (Docker, Kubernetes) let one kernel host many isolated workloads.
- Energy awareness: CPU frequency scaling, sleep states, especially in mobile.
- Distributed OS ideas: cloud OS layers (ChromeOS) offload storage/computation**.**

C++

What it is. C++ is a general-purpose, compiled, statically-typed programming language created by Bjarne Stroustrup (first released as "C with Classes" in 1985). It builds on C while adding abstractions for systems programming, application development, and high-performance code.

Core philosophy.

- Zero-overhead abstraction:you don't pay for features you don't use.
- Direct hardware access + high-level constructs.
- Deterministic resource control (RAII: Resource Acquisition Is Initialization).

Language features.

- **Multi-paradigm:** procedural (like C), object-oriented (classes, inheritance, polymorphism), generic (templates), and functional touches (lambdas, <algorithm>).
- **Type system:** strong static typing, user-defined types, const-correctness, references vs pointers, smart pointers (unique_ptr, shared_ptr).
- **Templates & STL:** compile-time generics (vector, map, unordered_map, algorithms like sort, transform). Template metaprogramming and, since C++20, concepts**.**
- **Memory model:** stack/heap, manual control when needed, but RAII + smart pointers reduce leaks. Move semantics (&&, std::move) avoid copies.
- **Concurrency**: threads (std::thread, std::async), mutexes, atomics, memory ordering (since C++11).

Typical toolchain:

- **Compiler:** GCC (g++), Clang, MSVC.
- **Build systems:** Make, CMake, Ninja**.**
- **Workflow:** write .cpp/.h → preprocess → compile to object files → link → executable.

Applications:

- Operating systems,
- game engines,
- real-time simulations,
- high-frequency trading,
- embedded systems,
- browsers,
- performance-critical libraries.

Modern tools.

- **C++11/14:** auto, lambdas, smart pointers, range-for.
- **C++17:** structured bindings, std::optional, filesystem.
- **C++20:** concepts, ranges, coroutines, modules (compiler support maturing).
- **C++23**: refinements (print, std::expected).
- **Style:** prefer RAII, avoid raw new/delete, use STL algorithms over hand-rolled loops**.**

DESKTOP PUBLISHING (DTP):

Definition:

Desktop publishing is the process of using software on a personal computer to create professional-looking documents that combine text, images, and layout—ready for print or digital distribution. It replaced manual paste-up in the 1980s (Aldus PageMaker + Apple LaserWriter sparked the DTP revolution).

Goals:

Give designers control over typography, page geometry, graphics, and color so the output is press-ready (or web-ready) without a traditional typesetting house.

Essential concepts.

- **Page layout**: master pages, grids/columns, margins, bleeds, slugs.
- **Typography**: fonts (serif/sans, OpenType features), kerning/tracking/leading, hyphenation & justification, styles (paragraph/character).
- **Color**: CMYK for print, RGB for screen, spot colors (Pantone), ICC color profiles, overprint/trapping.
- **Images**: resolution (300 dpi for print), vector vs raster, linking/embedding, preflight checks.
- **Output**: PDF/X standards for print, imposition (booklet/signatures), proofing, packaging files (fonts, links).

Common software & roles.

- **Professional layout**: Adobe InDesign (industry standard), Affinity Publisher, QuarkXPress.
- **Supporting tools**: Illustrator (vector art), Photoshop (raster), Acrobat (PDF preflight).
- **Office-level DTP**: Microsoft Publisher, Apple Pages, Canva (template-driven).
- **Open-source**: Scribus (layout), Inkscape (vector), GIMP (raster).

Typical workflow.

- **Plan:** page size, margins, grid; define style guide (fonts, colors).
- **Assemble:** place text (often from Word/Docs), flow into frames, place images/graphics.
- **Refine**: apply styles, adjust kerning, balance columns, check alignment.
- **Preflight & proof**: verify missing fonts/links, image resolution, color space, bleed.
- **Export**: PDF/X for printer or ePub/interactive PDF for digital.

APLICATIONS

- Brochures,
- magazines,
- books,
- newsletters,
- posters,
- packaging,
- annual reports,
- PDFs, and
- increasingly interactive digital publications.
- Good DTP balances aesthetic design with technical constraints (press specs, file size, readability).

INDUSTRIALVISIT SUMMARY

Institution: St. Philomena's (Autonomous) College, Mysore

Place Visited: University of Mysore

Date: [26-03-2026]

The students and faculty of St. Philomena's (Autonomous) College, Mysore, undertook an insightful industrial visit to the University of Mysore. The objective of the visit was to bridge the gap between theoretical knowledge and practical application in the field of computer science and data center.

During the internship we toured the university's central data center, the "heart" of campus IT services. The facility houses eight Lenovo servers serving distinct roles:

1. CCTV server (Lenovo IBM platform) – manages 180 RF cameras; protected by a dedicated firewall.

2. Backup servers – two units providing several terabytes of redundant storage.

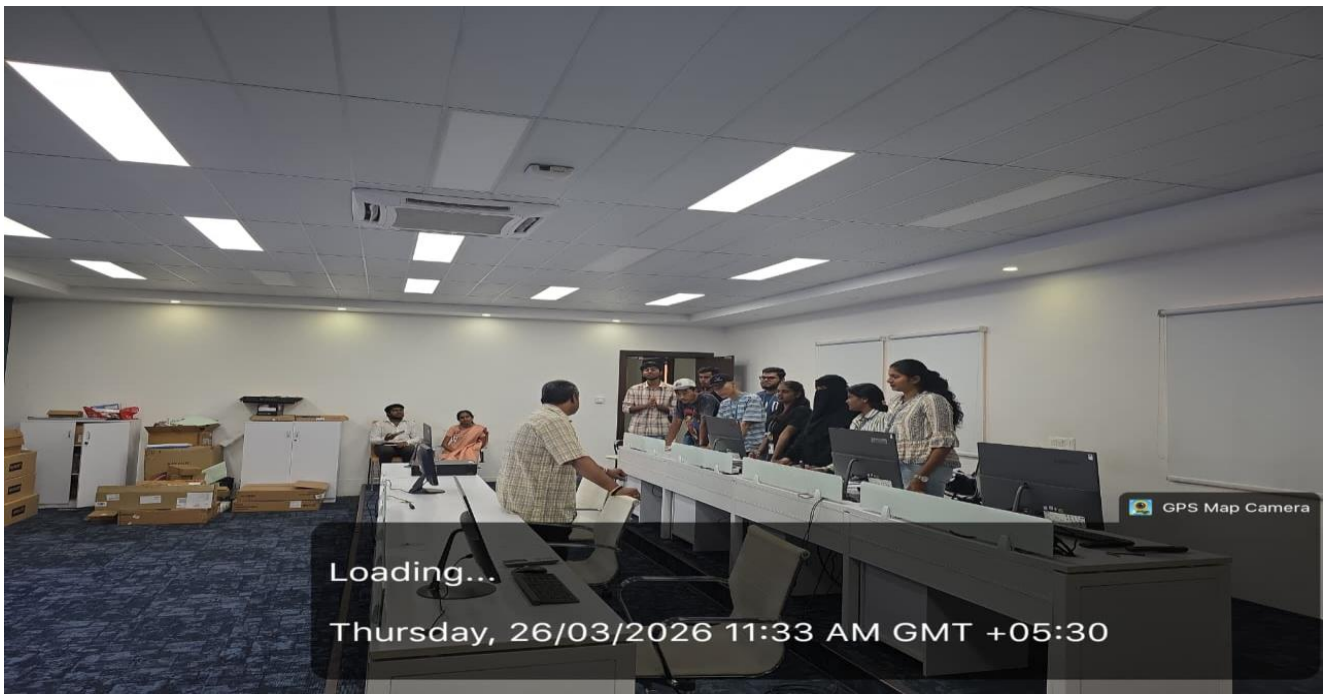
3. Internet server – links the DSM and MPC campus networks, offers VPN access for students and faculty, and connects via fiber-to-Cat6 media converters to a Layer-2 switch.

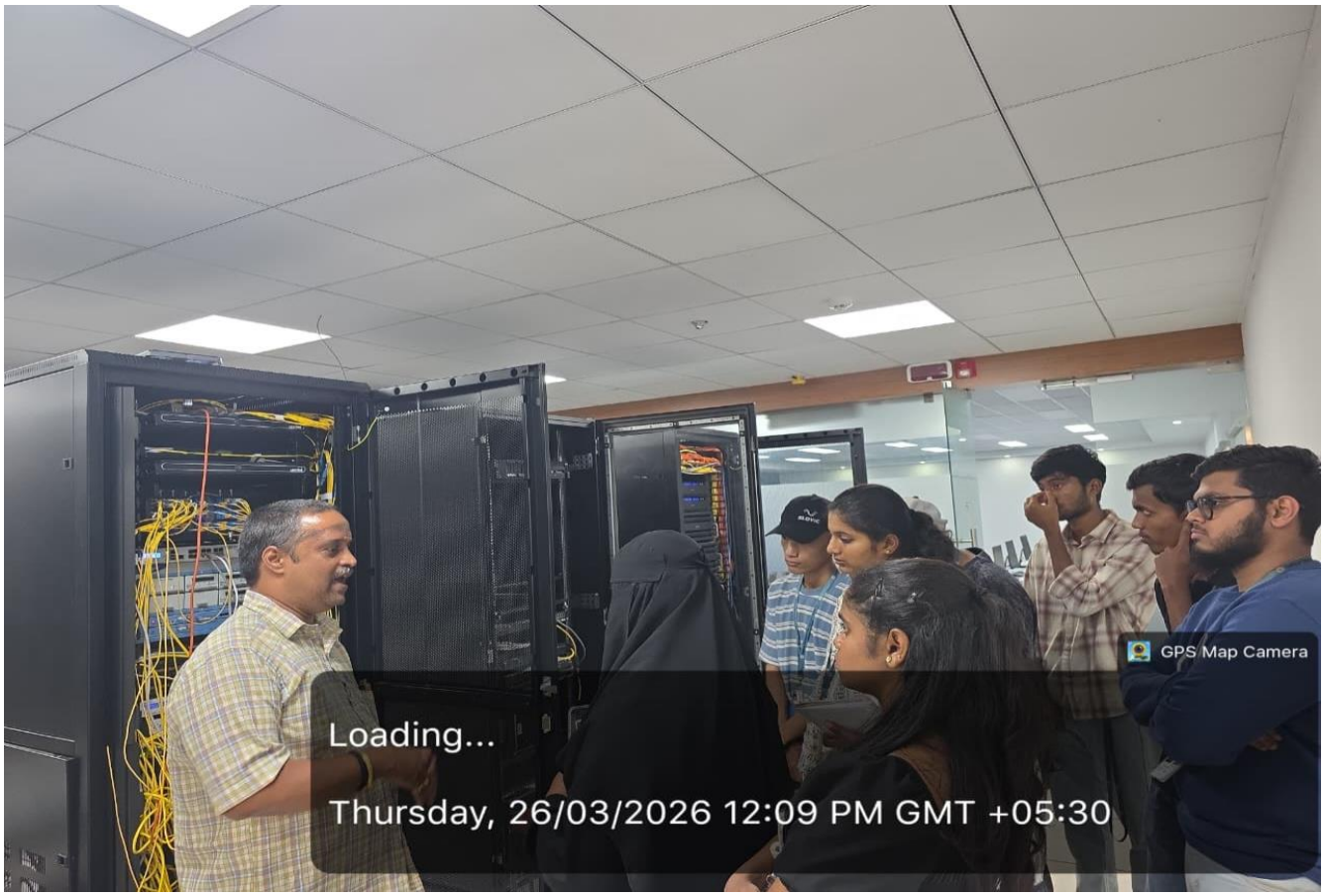
4. Data server– core repository protected by four firewalls; equipped with dual motherboards, dual processors, triple power feeds, climate control (18 °C), solar and battery-grid backup, and a 45-day data-retention policy.

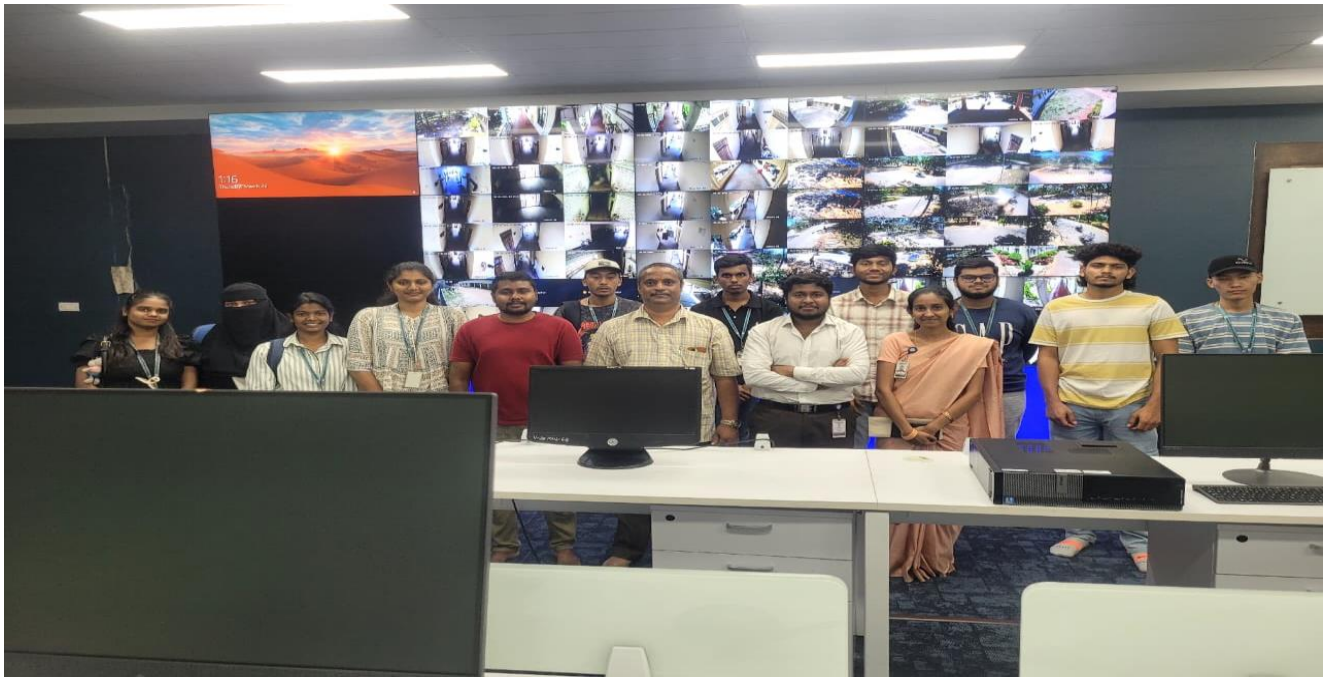
Network media: The campus fiber backbone uses color-coded single-mode OFC:

- Red (underground) – 12-core cable.
- Blue (underground & overhead) – 6-core cable.
- Green (overhead) – 16-core redundant cable, ensuring fail-over capacity.

The center supports the university website, 180 CCTV/RF cameras, and multiple academic services, operating under standard ISP connectivity (modem-based digital↔ analog conversion) and layered security per the OSI model.


GPS Map Camera
Loading...
Thursday, 26/03/2026 11:33 AM GMT +05:30


GPS Map Camera
Loading...
Thursday, 26/03/2026 12:09 PM GMT +05:30




GPS Map Camera
Loading...
Thursday, 26/03/2026 11:36 AM GMT +05:30

.

.